

SQLEquiv: A Multi-Method Library for SQL Query Equivalence Analysis

Algebraic, Graph-Based, and Embedding-Based Approaches

Yifan Yang

2026-04-17

Abstract

We present **SQLEquiv**, an open-source Python library that decides SQL query equivalence through three complementary lenses: rule-based relational-algebra rewriting, NetworkX-backed graph isomorphism with a weighted structural-similarity fallback, and cosine similarity over deterministic vector embeddings. A single unified analyzer dispatches to the three methods and reports a unanimous verdict together with per-method confidences. The paper formalizes what each method computes, details the software architecture, and works through a reproducible battery of examples — from textbook cases to queries with CTEs, window functions, set operations, and nested subqueries. We close with a documented set of known gaps and a forward-looking roadmap.

Table of contents

1	Introduction	1
2	Theory	2
2.1	Problem statement	2
2.2	Algebraic method	2
2.3	Graph method	2
2.4	Embedding method	3
2.5	Combining methods	3
3	Software architecture	3
3.1	Plugin registry	4
3.2	Dependency policy	4
4	Installation and reproducibility	4
5	Examples and simulations	5
5.1	Setting up	5
5.2	Identical queries	5
5.3	Structurally different queries	5
5.4	Common Table Expressions	6
5.5	Window functions	6
5.6	Known gaps	6
5.7	Custom analysis method (plugin)	7
5.8	Custom embedding model	7
5.9	Batch analysis	8
5.10	Latency micro-benchmark	8
5.11	Proof export	8

6	Reproducing the manuscript	9
7	Limitations and future work	9
	Acknowledgements	10
	References	10

1 Introduction

Deciding whether two SQL queries are **equivalent** — i.e. produce the same result for every database instance — is a foundational problem at the intersection of databases, programming languages, and modern NL2SQL tooling. In general, equivalence of SQL with aggregation, NULL semantics, and duplicates is undecidable (Chu et al. 2017), so practical systems combine conservative decision procedures (sound but incomplete) with heuristic similarity measures.

SQLEquiv is a modest open-source step in that direction. It exposes a single **SQLEquivalenceAnalyzer** that runs three independent methods and reports both a unanimous verdict and per-method confidences:

1. **Algebraic** — canonicalize each query into a relational-algebra expression tree and compare, driven by a fixed set of rewrite rules (selection / projection pushdown, join commutativity and associativity, union commutativity, etc.).
2. **Graph** — materialize a directed attribute graph from the query’s abstract syntax tree (AST) and run attribute-aware isomorphism, falling back to a weighted structural-similarity cascade when isomorphism fails.
3. **Embedding** — produce a deterministic vector per query (either via node-feature pooling over the graph, or a pluggable **encode**-bearing model such as the built-in **HashingVectorEncoder**) and score with cosine similarity mapped to $[0, 1]$.

The library targets Python 3.8+ and builds on [sqlglot](#) for dialect-robust parsing. This manuscript is a Quarto (Posit Software, PBC 2022) document so every code block is executed at render time — readers can reproduce every number, table, and figure below with the commands in Section 6.

2 Theory

2.1 Problem statement

Given two queries q_1, q_2 over a common database schema $\mathcal{D}$, write $\llbracket q \rrbracket_D$ for the multiset of rows returned by q on instance $D \in \mathcal{D}$. Two notions of equivalence are useful:

$$\begin{aligned} q_1 \equiv_{\text{set}} q_2 &\iff \forall D. \text{set}[\llbracket q_1 \rrbracket_D] = \text{set}[\llbracket q_2 \rrbracket_D] \\ q_1 \equiv_{\text{bag}} q_2 &\iff \forall D. \llbracket q_1 \rrbracket_D = \llbracket q_2 \rrbracket_D \end{aligned}$$

Set-equivalence ignores duplicates; bag-equivalence preserves multiplicities. The two differ on **UNION** vs **UNION ALL**, **SELECT** vs **SELECT DISTINCT**, and any rewrite that duplicates or collapses tuples. SQLEquiv’s algebraic checker targets bag-equivalence up to the supported rules; the graph and embedding checkers are structural heuristics that do not strictly decide either notion.

2.2 Algebraic method

The algebraic method compiles each query to a relational-algebra expression tree. We use the standard operator set $\{\pi, \sigma, \bowtie, \cup, \cap, -, \gamma, \tau, \delta\}$ (Silberschatz, Korth, and Sudarshan 2019; Garcia-Molina, Ullman, and Widom 2008): projection, selection, natural / theta join, union, intersect, difference, aggregation, sort, and distinct. The checker then tests equivalence in three phases:

1. **Identity** — direct structural equality $T_1 \stackrel{?}{=} T_2$ on the operator trees.
2. **Normalization** — rewrite each tree to a canonical form using idempotent transformations (Equation 1, Equation 2) and retest identity.
3. **Transformation search** — bounded DFS over the rewrite rules attempting to transform T_1 into T_2 within a configurable depth limit.

Selection pushdown:

$$\sigma_p(R \bowtie S) \equiv \begin{cases} \sigma_{p_R}(R) \bowtie \sigma_{p_S}(S) & \text{if } p = p_R \wedge p_S, \\ \sigma_{p_R}(R) \bowtie S & \text{if } p \text{ touches only } R, \\ R \bowtie \sigma_{p_S}(S) & \text{if } p \text{ touches only } S. \end{cases} \quad (1)$$

Join commutativity (inner join only):

$$R \bowtie S \equiv S \bowtie R. \quad (2)$$

When none of the phases produces a match, the checker returns a soft similarity in $[0, 1]$ built from operator-type distribution, tree structure, shared tables, and shared predicates.

2.3 Graph method

Every query is converted into a labeled directed graph $G = (V, E, \ell_V, \ell_E)$ where vertices correspond to AST nodes, edges encode parent-child AST relationships, and label functions ℓ_V, ℓ_E carry (**type**, **value**, **attributes**) dictionaries.

Two attribute-aware matchers are used:

$$\begin{aligned} \mu_V(u, v) &= [\ell_V(u).type = \ell_V(v).type] \cdot [\ell_V(u).value = \ell_V(v).value] \\ \mu_E(e_1, e_2) &= [\ell_E(e_1).type = \ell_E(e_2).type] \end{aligned}$$

Exact equivalence is reported when `networkx.is_isomorphic` returns true under (μ_V, μ_E) (Hagberg, Schult, and Swart 2008). Otherwise the checker computes a weighted similarity

$$\text{sim}_G(G_1, G_2) = \sum_{k \in \{\text{node}, \text{edge}, \text{struct}, \text{spec}\}} w_k \cdot s_k(G_1, G_2), \quad (3)$$

with weights $w = (0.3, 0.3, 0.2, 0.2)$ and components:

- s_{node} — blend of a Jaccard over the node-type histogram and a Jaccard over the (**type**, **value**) histogram (so a different table name actually lowers the score).
- s_{edge} — Jaccard over the edge-type histogram.
- s_{struct} — $1 - \|\vec{d}_1 - \vec{d}_2\|_2 / (\|\vec{d}_1\|_2 + \|\vec{d}_2\|_2)$ on sorted degree sequences.
- s_{spec} — same shape applied to Laplacian spectra.

The configurable threshold (default 0.9) maps Equation 3 to an APPROXIMATE verdict. For a hard distance, the library also exposes `compute_graph_edit_distance`, a wall-clock-bounded call to NetworkX’s `optimize_graph_edit_distance` (Abu-Aisheh, Raïsi, and Martineau 2015; Riesen and Bunke 2009).

2.4 Embedding method

Given a graph G and a per-node encoder $f_\theta : \mathcal{V} \rightarrow \mathbb{R}^d$, a pooled query vector is

$$\vec{v}(q) = \rho(\{f_\theta(n)\}_{n \in V}),$$

with ρ one of mean / sum / max pooling. The library ships two built-in encoders:

1. **NodeEmbedding** — deterministic pseudo-random vectors seeded by a stable hash of each node’s (**type**, **name**) key.
2. **HashingVectorEncoder** — a hashing-trick encoder (Weinberger et al. 2009) over the SQL text itself with optional n-gram features; zero-extra-dependency and reproducible.

Third-party models (GNNs, pretrained code LMs) plug in by exposing a single `encode(query) -> np.ndarray` method.

Similarity is cosine, clipped and rescaled to $[0, 1]$:

$$\text{sim}_E(q_1, q_2) = \max\left(0, \min\left(1, \frac{1 + \cos(\vec{v}(q_1), \vec{v}(q_2))}{2}\right)\right). \quad (4)$$

2.5 Combining methods

The analyzer runs each requested method and records per-method `{is_equivalent, confidence, equivalence_type}`. The headline verdict is **unanimous-agreement** — `is_equivalent` is true only when every method votes yes. The aggregate confidence is the fraction of yes votes, so disagreement surfaces as a fractional score. This policy is documented in `Future-Plan.md` as an open design question; callers who prefer weighted or short-circuit voting can compose their own logic over `AnalysisResult.method_results`.

3 Software architecture

The package is organised into seven subsystems:

Subsystem	Files	Responsibility
Parser	<code>parser/sql_parser.py</code> , <code>parser/ast_builder.py</code> , <code>parser/normalizer.py</code>	<code>sqlglot</code> $\rightarrow$ <code>ParsedQuery</code> + <code>ASTNode</code> tree.
Representations	<code>representations/algebraic/*</code> , <code>representations/graph/*</code> , <code>representations/embedding/*</code>	Algebraic trees, query graphs, logical query trees, vector encoders.
Equivalence	<code>equivalence/algebraic_equivalence.py</code> , <code>equivalence/graph_equivalence.py</code> , <code>equivalence/embedding_similarity.py</code>	<code>OneEqEquivalenceChecker</code> per equivalence method.
Operators	<code>operators/*.py</code>	<code>SELECT</code> / <code>FROM</code> / <code>JOIN</code> / aggregate / window / scalar operator classes.
Transformations	<code>transformations/algebraic_rules.py</code> , <code>transformations/graph_transformations.py</code> , <code>transformations/optimization_rules.py</code>	Rewrite rules consumed by the analyzer.
Analyzer	<code>analyzer.py</code> , <code>plugins.py</code> , <code>proof_export.py</code>	Public entry point, plugin registry, Markdown/JSON proof export.
Utils	<code>utils/*.py</code> , <code>models/embedding_models.py</code>	SQL / graph / algebra helpers, <code>HashingVectorEncoder</code> .

3.1 Plugin registry

Custom analysis methods are registered either at runtime or through the `sql_equivalence.methods` entry-point group. The plugin contract is a single callable (`parsed_query_1`, `parsed_query_2`, detailed:

bool) -> dict that returns at minimum `is_equivalent`, `confidence`, and `equivalence_type`. Built-in methods are themselves registered this way, so there is a single dispatch table.

3.2 Dependency policy

The core install depends only on `sqlglot`, `networkx`, `numpy`, `scipy`, `scikit-learn`, and `pandas`. Heavy visualization dependencies (`graphviz`, `plotly`) are imported lazily inside the functions that need them; the optional `[ml]` extra is reserved for plugin-authored learned encoders.

4 Installation and reproducibility

```
# Core install (no GPU / visualization deps)
pip install sql-equivalence

# Optional extras
pip install "sql-equivalence[viz]" # matplotlib + graphviz + plotly
pip install "sql-equivalence[ml]"  # torch + transformers (plugin targets)
pip install "sql-equivalence[dev]"  # pytest + ruff + mypy

# From source (developer mode)
git clone https://github.com/yfyang86/SQLEquiv
cd SQLEquiv
pip install -e "sql_equivalence[dev]"
```

To render this manuscript:

```
pip install -e . # or: pip install sql-equivalence
quarto render manuscript/main.qmd --to html # or --to pdf
```

5 Examples and simulations

All examples in this section are live Python blocks — the numbers below are produced by the current library at render time.

5.1 Setting up

```
import logging, warnings
logging.disable(logging.CRITICAL) # silence library chatter
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
from sql_equivalence import SQLEquivalenceAnalyzer
from sql_equivalence.models import HashingVectorEncoder
from sql_equivalence.proof_export import to_markdown

analyzer = SQLEquivalenceAnalyzer()
print("Available methods:", analyzer.available_methods)
```

Available methods: ('algebraic', 'embedding', 'graph')

5.2 Identical queries

```
sql = "SELECT id, name FROM users WHERE age >= 18"
result = analyzer.analyze(sql, sql)
print(f"Verdict: {result.is_equivalent} (confidence {result.confidence:.2f})")
for method, outcome in result.method_results.items():
    print(f"    {method}: eq={outcome['is_equivalent']} "
          f"    f"conf={outcome['confidence']:.3f}")
```

```
Verdict: True (confidence 1.00)
  algebraic: eq=True conf=1.000
  graph: eq=True conf=1.000
  embedding: eq=True conf=1.000
```

5.3 Structurally different queries

```
pairs = [
    ("Same query, different DB table",
     "SELECT id FROM users", "SELECT id FROM customers"),
    ("Plain vs aggregated",
     "SELECT id FROM t", "SELECT dept, COUNT(*) FROM emp GROUP BY dept HAVING COUNT(*) > 1"),
    ("Extra JOIN",
     "SELECT a.id FROM a JOIN b ON a.id = b.aid JOIN c ON b.cid = c.id",
     "SELECT a.id FROM a JOIN b ON a.id = b.aid"),
]
rows = []
for name, q1, q2 in pairs:
    r = analyzer.analyze(q1, q2)
    rows.append({
        "case": name,
        "overall_is_equivalent": r.is_equivalent,
        "confidence": round(r.confidence, 3),
        "algebraic": r.method_results["algebraic"]["is_equivalent"],
        "graph": r.method_results["graph"]["is_equivalent"],
        "embedding": r.method_results["embedding"]["is_equivalent"],
    })
pd.DataFrame(rows)
```

	case	overall_is_equivalent	confidence	algebraic	graph	embedding
0	Same query, different DB table	False	0.667	True	False	True
1	Plain vs aggregated	False	0.000	False	False	False
2	Extra JOIN	False	0.667	True	False	True

5.4 Common Table Expressions

```
cte1 = ("WITH adults AS (SELECT id, name FROM users WHERE age >= 18) "
        "SELECT id FROM adults")
cte2 = ("WITH adults AS (SELECT id, name FROM users WHERE age >= 18) "
        "SELECT id FROM adults")
print("Identical CTE:", analyzer.analyze(cte1, cte2).is_equivalent)
```

```
Identical CTE: True
```

5.5 Window functions

```
w1 = ("SELECT id, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) "  
      "AS rn FROM emp")  
w2 = ("SELECT id, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) "  
      "AS rn FROM emp")  
r = analyzer.analyze(w1, w2)  
print(f"Identical window fn: eq={r.is_equivalent} conf={r.confidence:.2f}")
```

Identical window fn: eq=True conf=1.00

5.6 Known gaps

The following cases are deliberately tracked as `xfail(strict=True)` regression markers in `sql_equivalence/tests/fixtures`. The analyzer currently returns **equivalent** for each; tightening any entry here requires the corresponding checker rule to be tightened.

```
gaps = [  
    ("Opposite WHERE",  
     "SELECT id FROM users WHERE age > 10",  
     "SELECT id FROM users WHERE age < 10"),  
    ("INNER vs LEFT JOIN",  
     "SELECT a.id FROM a INNER JOIN b ON a.id = b.aid",  
     "SELECT a.id FROM a LEFT JOIN b ON a.id = b.aid"),  
    ("UNION vs UNION ALL",  
     "SELECT id FROM a UNION SELECT id FROM b",  
     "SELECT id FROM a UNION ALL SELECT id FROM b"),  
    ("LIMIT 10 vs LIMIT 100",  
     "SELECT id FROM t LIMIT 10",  
     "SELECT id FROM t LIMIT 100"),  
    ("COUNT vs SUM",  
     "SELECT dept, COUNT(*) FROM emp GROUP BY dept",  
     "SELECT dept, SUM(salary) FROM emp GROUP BY dept"),  
]  
pd.DataFrame([  
    {"case": n, "overall_is_equivalent": analyzer.analyze(a, b).is_equivalent}  
    for n, a, b in gaps  
])
```

	case	overall_is_equivalent
0	Opposite WHERE	True
1	INNER vs LEFT JOIN	True
2	UNION vs UNION ALL	True
3	LIMIT 10 vs LIMIT 100	True
4	COUNT vs SUM	True

5.7 Custom analysis method (plugin)

```
from sql_equivalence import plugins  
  
def identical_text_only(parsed1, parsed2, detailed):  
    eq = parsed1.sql.strip() == parsed2.sql.strip()  
    return {
```

```

        "is_equivalent": eq,
        "confidence": 1.0 if eq else 0.0,
        "equivalence_type": "exact" if eq else "not_equivalent",
    }

plugins.unregister_method("text_eq")
analyzer.register_method("text_eq", identical_text_only)
r = analyzer.analyze(
    "SELECT id FROM t",
    "select id from t",
    methods=["text_eq"],
)
print("Plugin vote:", r.method_results["text_eq"])

```

Plugin vote: {'is_equivalent': False, 'confidence': 0.0, 'equivalence_type': 'not_equivalent'}

5.8 Custom embedding model

```

encoder = HashingVectorEncoder(dim=128, ngram_range=(1, 2))
analyzer_custom = SQLEquivalenceAnalyzer(embedding_model=encoder)
r = analyzer_custom.analyze(
    "SELECT id, name FROM users",
    "select id , name from users",
    methods=["embedding"],
)
print(f"Whitespace/case only: sim={r.method_results['embedding']['confidence']:.3f}")

```

Whitespace/case only: sim=1.000

5.9 Batch analysis

```

batch = [
    ("SELECT id FROM t", "SELECT id FROM t"),
    ("SELECT id FROM users", "SELECT id FROM customers"),
    ("SELECT * FROM a", "SELECT dept, COUNT(*) FROM emp GROUP BY dept"),
]
results = analyzer.batch_analyze(batch)
pd.DataFrame([
    {"sql1": r.sql1, "sql2": r.sql2,
     "is_equivalent": r.is_equivalent,
     "confidence": round(r.confidence, 3)}
    for r in results
])

```

	sql1	sql2	is_equivalent	confidence
0	SELECT id FROM t	SELECT id FROM t	True	1.000
1	SELECT id FROM users	SELECT id FROM customers	False	0.667
2	SELECT * FROM a	SELECT dept, COUNT(*) FROM emp GROUP BY dept	False	0.000

5.10 Latency micro-benchmark

A rough end-to-end latency profile on the shared fixture corpus (run on whatever machine renders this document):


```

import statistics, time
from sql_equivalence.tests.fixtures import EQUIVALENT_PAIRS, NON_EQUIVALENT_PAIRS

CORPUS = list(EQUIVALENT_PAIRS) + list(NON_EQUIVALENT_PAIRS)
samples_ms = []
bench_analyzer = SQLEquivalenceAnalyzer(enable_caching=False)
for _ in range(10):
    for sql1, sql2 in CORPUS:
        t = time.perf_counter()
        bench_analyzer.analyze(sql1, sql2)
        samples_ms.append((time.perf_counter() - t) * 1000.0)

pd.DataFrame([
    "pairs": len(CORPUS),
    "runs": 10,
    "min_ms": round(min(samples_ms), 2),
    "mean_ms": round(statistics.mean(samples_ms), 2),
    "median_ms": round(statistics.median(samples_ms), 2),
    "max_ms": round(max(samples_ms), 2),
])

```

	pairs	runs	min_ms	mean_ms	median_ms	max_ms
0	11	10	1.56	2.75	2.69	4.42

5.11 Proof export

```

r = analyzer.analyze(
    "SELECT id FROM t",
    "SELECT dept, COUNT(*) FROM emp GROUP BY dept HAVING COUNT(*) > 1",
    methods=["algebraic"],
    detailed=True,
)
print(to_markdown(r)[:1200])

```

SQL equivalence analysis

****Verdict**:** NOT EQUIVALENT (confidence 0.000)
****Execution time**:** 8.23 ms

Queries

```

```sql
-- Q1
SELECT id FROM t
```
```sql
-- Q2
SELECT dept, COUNT(*) FROM emp GROUP BY dept HAVING COUNT(*) > 1
```

```

Per-method results

```

- **algebraic** - type=not_equivalent, confidence=0.500
  - `canonical_form1`: [*](unknown)

```

```
- `canonical_form2`: [*] ( [{}] (G[] (unknown)))
```

6 Reproducing the manuscript

The entire document is an executable Quarto notebook. To reproduce:

```
# 1. Install the library and manuscript dependencies
pip install sql-equivalence quarto-cli jupyter pandas

# 2. Render the manuscript to HTML or PDF
quarto render manuscript/main.qmd --to html
quarto render manuscript/main.qmd --to pdf    # requires LaTeX
```

Every numeric value and every table in Section 5 is produced by the live code blocks at render time. The corpus lives in `sql_equivalence/tests/fixtures.py`; swapping in a larger corpus is a one-line edit.

7 Limitations and future work

SQLEquiv is an alpha-stage library and the known gaps listed in Section 5 are real. The algebraic checker in particular is too lenient on column-name and predicate-direction differences because its canonical form collapses to a `Project(*, Relation(t))` scaffold when AST traversal bottoms out. Tightening requires: (i) projection column-list comparison, (ii) predicate-sign aware normalization, and (iii) multiplicity-aware rules for `UNION` vs `UNION ALL` and `DISTINCT`. The graph similarity weights are set by hand; learning them from a labeled corpus is an obvious avenue for future work.

Scope boundaries that we do **not** plan to cross are spelled out in `Future-Plan.md`: we will not ship a custom SQL parser (sqlglot is the source of truth), we will not execute queries against a live database to sample equivalence, and we will not grow the default install with GPU-dependent models. Anything richer belongs in a plugin package.

Acknowledgements

The library is part of the open NL2SQL / Chat2BI course (2025-Dec). SQL parsing is carried by `sqlglot` (Mao 2020); graph algorithms use `networkx` (Hagberg, Schult, and Swart 2008); embeddings leverage `numpy` and `scikit-learn`.

References

- Abu-Aisheh, Zeina, Romain Raïsi, and Patrick Martineau. 2015. “A Survey of Graph Edit Distance.” *Pattern Analysis and Applications* 18 (1): 1–20.
- Chu, Shumo, Konstantin Weitz, Alvin Cheung, and Dan Suciu. 2017. “Cosette: An Automated Prover for SQL.” In *Conference on Innovative Data Systems Research (CIDR)*.
- Garcia-Molina, Hector, Jeffrey D. Ullman, and Jennifer Widom. 2008. *Database Systems: The Complete Book*. 2nd ed. Prentice Hall.
- Hagberg, Aric A., Daniel A. Schult, and Pieter J. Swart. 2008. “Exploring Network Structure, Dynamics, and Function Using NetworkX.” In *Proceedings of the 7th Python in Science Conference (SciPy)*, 11–15.
- Mao, Toby. 2020. “Sqlglot: SQL Parser, Transpiler, and Optimizer.” <https://github.com/tobymao/sqlglot>.
- Posit Software, PBC. 2022. “Quarto: An Open-Source Scientific and Technical Publishing System.” <https://quarto.org/>.
- Riesen, Kaspar, and Horst Bunke. 2009. “Approximate Graph Edit Distance Computation by Means of Bipartite Graph Matching.” *Image and Vision Computing* 27 (7): 950–59.
- Silberschatz, Abraham, Henry F. Korth, and S. Sudarshan. 2019. *Database System Concepts*. 7th ed. McGraw-Hill.

Weinberger, Kilian, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. 2009. “Feature Hashing for Large Scale Multitask Learning.” In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, 1113–20.